Assignment Report

Name: Asser Mohammed Khairallah
Group Code: GIZ5_SWD5_S2
Assignment: MVC03 - ViewModels, Architectural Boundaries, and Data Transfer Security

---

Part 01: Theoretical Questions

1. State 3 mandatory architectural cases where using a ViewModel is strictly required over passing Domain Entities directly to the View in ASP.NET Core MVC.

- **Case 1: Security (Over-Posting / Mass Assignment and Schema Exposure):** The MVC model binder uses reflection to match incoming form fields, query strings, and route values to the public settable properties of the action parameter type. If a Domain Entity (e.g., `Employee`) is used directly as the parameter, an attacker can add extra fields to the HTTP POST request (using browser dev tools or tools like Postman), such as `IsAdmin=true`, `Salary=999999`, or `Id=5`. The binder will populate these properties even though the form never displayed them, and a subsequent `context.Update(entity)` or `SaveChanges()` would persist the tampered values. This is the Over-Posting (Mass Assignment) vulnerability. A ViewModel mitigates it by acting as an explicit whitelist: it contains only the properties the form is allowed to modify (e.g., `Name`, `Email`), so any extra fields in the request have no matching property and are simply ignored. The controller then maps the ViewModel to the entity manually (or via AutoMapper), keeping full control over which fields reach the database. In the opposite direction, passing entities to the View also leaks the database schema (internal IDs, password hashes, audit columns like `CreatedBy`, soft-delete flags, and navigation properties) into the rendered HTML or serialized JSON. A ViewModel exposes only the columns the UI actually needs.
- **Case 2: Data Shaping and Composition (Multiple Sources in One View):** A single Razor View often needs data that does not map to one entity, such as an employee's details plus a `SelectList` of departments for a dropdown, or aggregated values like `TotalSalary`. The `@model` directive accepts only one type, so a ViewModel is required to compose data from multiple entities and UI-only helper objects into a single strongly typed shape, without polluting the entity with `[NotMapped]` properties.
- **Case 3: View-Specific Validation and Presentation Rules:** Fields like `ConfirmPassword`, `[Compare]`, `[DataType]`, or display formatting attributes belong to the UI contract and have no corresponding database column. Placing them on an entity mixes UI concerns into the persistence model and can force EF Core to treat them as mapped columns. A ViewModel keeps UI validation attributes and presentation-only properties separate from the entity's database mapping.

Part 02: Additional Concepts

1. Explain how ViewModels enforce Separation of Concerns (SoC) and prevent tight coupling between the Persistence Layer (EF Core Entities) and the Presentation Layer (Razor Views) during the MVC request-response lifecycle.

- **Request flow (inbound):** The model binder creates the ViewModel instance from the HTTP request and validates it against its data annotations. The controller acts as the boundary: it maps the validated ViewModel to a Domain Entity, which is the only object EF Core's change tracker ever sees. Raw HTTP input never reaches the persistence layer directly.
- **Response flow (outbound):** The controller queries entities through EF Core, projects or maps

them into a ViewModel, and passes only that ViewModel to `View()`. The Razor View is compiled against the ViewModel type (`@model`), so it has no compile-time dependency on the entity class or on EF Core.

- **Prevents tight coupling:** Because the View depends only on the ViewModel, the database schema can change (renamed columns, split tables, new relationships) without modifying any Razor View. Only the mapping code in the controller or service layer changes. Likewise, UI changes do not force migrations or schema changes.
- **Prevents persistence leaks into the UI:** Passing tracked entities to the View risks accidental lazy-loading triggers during rendering (producing extra queries or N+1 problems), exposure of navigation properties, and circular reference problems during JSON serialization. A ViewModel is a plain, flat object with no change-tracking state, so none of these persistence behaviors can leak into the Presentation Layer.